

```

15
16 <script>
17   function mostrarIdiomes() {
18     const select = document.getElementById("idiomes");
19     const seleccionats = [];
20
21     for (let i = 0; i < select.options.length; i++) {
22       if (select.options[i].selected) {
23         seleccionats.push(select.options[i].text);
24       }
25     }
26
27     const resultat = document.getElementById("resumIdiomes");
28     if (seleccionats.length === 0) {
29       resultat.textContent = "No has seleccionat cap idioma.";
30     } else {
31       resultat.innerHTML = '<strong>Idiomes seleccionats:</strong> ${seleccionats.join(", ")}';
32     }
33   }
34 </script>

```

2.2 Validació de formularis en el cantó del client

La **validació de formularis** garanteix que les dades introduïdes per l'usuari siguin **coherents i segures** abans de processar-les o enviar-les al servidor.

Podem combinar **atributs HTML5** amb **JavaScript** per aconseguir una validació més rica i controlada.

2.2.1 Tipus d'elements que podem validar

Els formularis poden contenir **molts tipus de camps diferents**, i cadascun d'ells serveix per recollir un tipus d'informació concreta.

A la taula 4.1 en veiem alguns dels més habituals:

TAULA 2.1. Tipus d'elements que podem validar

Tipus d'element	Exemple	Funció principal
Text (text)	<input type="text">	Introduir text general (nom, cognoms, etc.)
Correu (email)	<input type="email">	Assegura que el format sigui de correu electrònic
Nombre (number)	<input type="number">	Permet introduir números amb límits mínim i màxim
Àrea de text (textarea)	<textarea></textarea>	Permet escriure textos llargs o missatges
Selecció única (radio)	<input type="radio">	Permet triar una sola opció entre diverses

TAULA 2.1 (continuació)

Tipus d'element	Exemple	Funció principal
Selecció múltiple (checkbox)	<code><input type="checkbox"></code>	Permet activar o desactivar opcions independents
Llista desplegable (select)	<code><select>...</select></code>	Permet triar opcions d'una llista predefinida
Botó (button)	<code><button></button></code>	Envia el formulari o executa accions

N'hi ha molts més tipus (date, color, range, file, tel, etc.), però aquests són els que trobem més sovint en aplicacions web bàsiques i són ideals per practicar la **validació de formularis**.

Cas pràctic: Exemple complet amb validacions amb JavaScript

En aquest formulari inventarem algunes **validacions bàsiques** per comprovar:

- Que el **nom** tingui almenys 3 caràcters.
- Que el **correu** contingui un @ i un punt ..
- Que l'**edat** estigui entre 18 i 99 anys.
- Que s'hagi seleccionat un **gènere**.
- Obtindrem les possibles **aficions** i validarem que almenys n'hi hagi una de seleccionada.
- Que s'hagin **acceptat les condicions**.

Si totes aquestes condicions es compleixen, mostrarem un missatge de validació correcta.

```
1 <!DOCTYPE html>
2 <html lang="ca">
3 <head>
4   <meta charset="UTF-8">
5   <title>Formulari complet amb validació</title>
6   <style>
7     form {
8       border: 1px solid #ccc;
9       padding: 15px;
10      width: 320px;
11    }
12    label {
13      display: block;
14      margin-top: 10px;
15    }
16    #missatge {
17      margin-top: 10px;
18      font-weight: bold;
19    }
20  </style>
```

```
21 </head>
22 <body>
23
24   <h3>Formulari complet amb validació bàsica</h3>
25
26   <form id="formulariComplet" onsubmit="validarFormulari();
27     return false;">
28     <label for="nom">Nom complet:</label>
29     <input type="text" id="nom" name="nom">
30
31     <label for="email">Correu electrònic:</label>
32     <input type="email" id="email" name="email">
33
34     <label for="edat">Edat:</label>
35     <input type="number" id="edat" min="18" max="99">
36
37     <p>Gènere:</p>
38     <label><input type="radio" name="genere" value="home"> Home
39       </label>
40     <label><input type="radio" name="genere" value="dona"> Dona
41       </label>
42
43     <label for="aficions">Aficions (pots seleccionar més d'una)
44       </label>
45     <select id="aficions" multiple size="4">
46       <option value="musica">Música</option>
47       <option value="esport">Esport</option>
48       <option value="viatges">Viatges</option>
49       <option value="lectura">Lectura</option>
50     </select>
51
52     <label><input type="checkbox" id="condicions"> Accepto les
53       condicions</label>
54
55     <button type="submit">Enviar</button>
56     <p id="missatge"></p>
57   </form>
58
59   <script>
60     function validarFormulari() {
61       const nom = document.getElementById("nom").value.trim();
62       const email = document.getElementById("email").value.trim
63         ();
64       const edat = document.getElementById("edat").value;
65       const genere = document.querySelector("input[name='genere
66         ']:checked");
67       const condicions = document.getElementById("condicions").
68         checked;
69       const missatge = document.getElementById("missatge");
70       const selectAficions = document.getElementById("aficions")
71         ;
72       const aficionsSeleccionades = [];
73
74       // Obtenim les aficions seleccionades
75       for (let i = 0; i < selectAficions.options.length; i++) {
76         if (selectAficions.options[i].selected) {
77           aficionsSeleccionades.push(selectAficions.options[i].
78             value);
79         }
80       }
81
82       // Validació del nom
83       if (nom.length < 3) {
```

```
74     missatge.textContent = "El nom ha de tenir almenys 3 car
       àcters.";
75     missatge.style.color = "red";
76     return;
77 }
78
79 // Validació del correu electrònic
80 if (!email.includes("@") || !email.includes(".")) {
81     missatge.textContent = "Introdueix un correu electrònic
       vàlid.";
82     missatge.style.color = "red";
83     return;
84 }
85
86 // Validació de l'edat
87 if (edat < 18 || edat > 99 || edat === "") {
88     missatge.textContent = "L'edat ha d'estar entre 18 i 99
       anys.";
89     missatge.style.color = "red";
90     return;
91 }
92
93 // Validació del gènere
94 if (!genere) {
95     missatge.textContent = "Selecciona un gènere.";
96     missatge.style.color = "red";
97     return;
98 }
99
100 // Validació de les aficions
101 if (aficionsSeleccionades.length === 0) {
102     missatge.textContent = "Selecciona almenys una afició.";
103     missatge.style.color = "red";
104     return;
105 }
106
107 // Validació de la checkbox
108 if (!condicions) {
109     missatge.textContent = "Has d'acceptar les condicions
       per continuar.";
110     missatge.style.color = "red";
111     return;
112 }
113
114 // Si tot és correcte
115 missatge.textContent = "Formulari correcte! Totes les
       dades són vàlides.";
116 missatge.style.color = "green";
117 }
118 </script>
119
120 </body>
121 </html>
```

Aquestes validacions no són “oficials” del navegador, sinó **normes que nosaltres inventem amb JavaScript** per comprovar si les dades tenen sentit abans d'enviar-les:

- **Nom:** evitem que l'usuari deixi el camp gairebé buit.
- **Correu electrònic:** comprovem si té la forma d'un correu, tot i que la

validació és molt senzilla (en realitat, els correus poden ser més complexos).

- **Edat:** ens assegurem que sigui un número dins d'un rang raonable.
- **Gènere:** evitem que s'envii sense cap opció marcada.
- **Aficions:** comprovem que l'usuari com a mínim hagi seleccionat una opció.
- **Condicions:** garantim que l'usuari les accepta abans d'enviar.

Aquestes comprovacions són habituals en formularis reals perquè eviten errors i dades incompletes.

Més endavant, quan aprenguem esdeveniments i el mètode `addEventListener()`, escriurem aquest mateix codi d'una manera més professional i separant completament l'HTML del JavaScript. Per ara, però, l'ús de `onsubmit` amb `return false` és **la forma més senzilla i visual** per practicar i entendre com funciona la validació.

Els esdeveniments i el mètode `addEventListener()`, es veuran amb deteniment en la Unitat 3.

Cas pràctic: Exemple de validacions sense ús de Javascript

Amb atributs propis de HTML5 també podem validar formularis de forma semblant a com ho hem fet amb JavaScript.

Anem a veure el mateix exemple i posteriorment analitzem quina és la millor solució:

```

1 <!DOCTYPE html>
2 <html lang="ca">
3   <head>
4     <meta charset="UTF-8" />
5     <title>Formulari amb validació HTML5</title>
6     <style>
7       form { border: 1px solid #ccc; padding: 15px; width: 320px; }
8       label { display: block; margin-top: 10px; }
9       /* Feedback visual natiu amb CSS */
10      input:required:invalid, textarea:required:invalid, select:
11        required:invalid {
12        border: 2px solid #e57373; /* vermell suau */
13      }
14      input:required:valid, textarea:required:valid, select:
15        required:valid {
16        border: 2px solid #81c784; /* verd suau */
17      }
18      small.hint { color: #666; font-size: 0.9em; }
19    </style>
20  </head>
21  <body>
22    <h3>Formulari complet amb validació HTML5</h3>
23
24    <!-- Sense JS: el navegador valida abans d'enviar -->
25    <form id="formulariComplet" action="#" method="post">
26      <!-- NOM: mínim 3 caràcters, només lletres i espais -->
27      <label for="nom">Nom complet:</label>

```

```
26 <input
27   type="text"
28   id="nom"
29   name="nom"
30   required
31   minlength="3"
32   pattern="^[A-Za-zÀÿ-\s'\-]{3,}$"
33   autocomplete="name"
34   title="Mínim 3 caràcters. Només lletres, espais, apò
      strof i guionet."
35   placeholder="Maria Pujol"
36 />
37 <small class="hint">Ex.: Maria Pujol</small>
38
39 <!-- EMAIL: format de correu -->
40 <label for="email">Correu electrònic:</label>
41 <input
42   type="email"
43   id="email"
44   name="email"
45   required
46   autocomplete="email"
47   inputmode="email"
48   placeholder="exemple@correu.com"
49   title="Introdueix un correu vàlid (exemple@correu.com)."
```

```
50 />
51
52 <!-- EDAT: entre 18 i 99 -->
53 <label for="edat">Edat:</label>
54 <input
55   type="number"
56   id="edat"
57   name="edat"
58   required
59   min="18"
60   max="99"
61   inputmode="numeric"
62   placeholder="18"
63   title="Edat entre 18 i 99."
64 />
65
66 <!-- GÈNERE: radio (posem required a un del grup) -->
67 <p>Gènere: <small class="hint">(obligatori)</small></p>
68 <label
69   ><input type="radio" name="genere" value="home" required
      /> Home</label>
70 >
71 <label><input type="radio" name="genere" value="dona" />
      Dona</label>
72
73 <!-- AFICIONS: multiselect obligatori -->
74 <label for="aficions">Aficions (selecciona almenys una):</
      label>
75 <select
76   id="aficions"
77   name="aficions"
78   multiple
79   size="4"
80   required
81   title="Selecciona almenys una afició."
82 >
83   <option value="musica">Música</option>
```

```
84     <option value="esport">Esport</option>
85     <option value="viatges">Viatges</option>
86     <option value="lectura">Lectura</option>
87 </select>
88 <small class="hint">Prem Ctrl (o Cmd) per seleccionar més
    d'una.</small>
89
90 <!-- CONDICIONS: checkbox obligatori -->
91 <label style="margin-top:10px;">
92     <input type="checkbox" id="condicions" name="condicions"
        required />
93     Accepto les condicions
94 </label>
95
96     <button type="submit" style="margin-top:10px;">Enviar</
        button>
97 </form>
98
99 <p style="margin-top:12px;">
100     Amb aquesta configuració, si falten camps o tenen format
        incorrecte, el
101     navegador mostrarà missatges d'error nadius i impedirà l'
        enviament fins
102     que tot sigui vàlid.
103 </p>
104 </body>
105 </html>
```

Què hem fet amb HTML5 (sense JavaScript)?

- **required:** fa que el camp sigui obligatori (bloqueja l'enviament si està buit).
- **type="email":** activa la comprovació bàsica de correu.
- **min/max/minlength:** limiten edats i longituds mínimes.
- **pattern:** expressió regular per controlar el format (ex.: nom amb lletres i espais).
- **title:** text d'ajuda que el navegador pot mostrar quan la validació falla.
- **Radio:** n'hi ha prou amb posar **required** a un dels input del grup perquè el navegador exigeixi que n'hi hagi un marcat.
- **Checkbox:** amb **required** forces que s'hagi de marcar per poder enviar.

2.2.2 Validar amb HTML5 o amb JavaScript

Quan treballem amb formularis, tenim **dues grans maneres de validar les dades** abans d'enviar-les:

- fer-ho **amb HTML5**, aprofitant els atributs i comportaments nadius del navegador, o bé

- fer-ho **amb JavaScript**, que ens dona més control i personalització.

Totes dues opcions són vàlides i sovint **es combinen** en projectes reals: primer deixem que HTML5 faci la seva feina bàsica, i després afegim una validació JavaScript per cobrir casos més específics.

Vegem amb detall els avantatges i limitacions de cadascuna.

Validar amb HTML5

Els navegadors moderns ja incorporen un sistema de validació automàtica que s'activa només amb l'ús de determinats **atributs** (required, type, min, max, pattern, step, minlength, maxlength, etc.).

Aquest sistema s'anomena **Constraint Validation API**, i permet bloquejar l'enviament del formulari i mostrar missatges d'error natius sense necessitat de programar res en JavaScript.

Avantatges:

- **Simplicitat total.** No cal escriure codi JavaScript per comprovar si els camps estan buits o tenen un format incorrecte. Només cal afegir els atributs adequats i el navegador fa la resta. Això ja assegura que l'usuari no pugui enviar el formulari sense complir aquestes condicions.

```
1 <input type="email" required>  
2 <input type="number" min="18" max="99">
```

- **Missatges automàtics.** Els navegadors mostren missatges d'avís personalitzats segons l'idioma del sistema, com ara *"Aquest camp és obligatori"* o *"Introdueix una adreça de correu electrònic vàlida"*. Això redueix l'esforç de desenvolupament i ajuda a mantenir una interfície coherent.
- **Menys codi i menys errors.** En projectes senzills (formularis de contacte, subscripcions, registres bàsics) la validació HTML5 és suficient, segura i estalvia temps. Menys codi vol dir també menys possibilitats d'errors de lògica.
- **Compatibilitat molt bona.** Tots els navegadors moderns (Chrome, Firefox, Edge, Safari, Opera) suporten la majoria d'aquests atributs i comportaments de validació, de manera consistent.
- **Feedback visual natiu.** El navegador també aplica estils visuals com el contorn vermell als camps invàlids o verd als correctes, i permet personalitzar-los amb CSS (:invalid, :valid).

Limitacions:

- **Personalització limitada dels missatges.** Tot i que podem traduir-los parcialment amb `title` o amb l'API de validació (`setCustomValidity()`), els missatges d'error són bàsicament els que decideix el navegador. No podem, per exemple, mostrar un text d'avís al costat d'un camp amb un estil concret sense afegir codi JavaScript.
- **No permet validacions complexes entre camps.** Per exemple, comprovar si la contrasenya i la confirmació coincideixen, o si una data d'inici és anterior a una data de finalització. Aquestes situacions requereixen lògica condicional que només podem fer amb JavaScript.
- **Dependència de l'experiència del navegador.** Cada navegador pot mostrar missatges o icones lleugerament diferents, cosa que pot afectar la coherència visual de la web. Per aplicacions amb disseny molt personalitzat o corporatiu, això pot ser un inconvenient.

Validar amb JavaScript

La validació amb JavaScript és més potent i flexible.

Ens permet controlar completament **quan, com i on** mostrem els missatges d'error, i podem definir **les nostres pròpies regles** sense dependre del navegador.

Avantatges:

- **Personalització total.** Podem crear missatges específics per a cada cas, adaptar-los al llenguatge del projecte o mostrar-los visualment d'una manera atractiva (quadres de colors, icones, etc.). Per exemple:

```
1 if (!email.includes("@")) {  
2   missatge.textContent = "El correu ha de contenir un '@'.";  
3 }
```

- **Control sobre la interfície.** No cal que el navegador mostri els seus missatges nadius: podem decidir quan i com avisar l'usuari. Això ens permet, per exemple, mostrar tots els errors junts al final del formulari o marcar en vermell només els camps concrets amb problemes.
- **Validacions entre camps.** Amb JavaScript podem fer comprovacions que HTML5 no permet, com assegurar que:
 - la contrasenya i la confirmació coincideixin,
 - la data d'inici sigui anterior a la data de fi,
 - o que s'hagi triat almenys una afició d'una llista de checkbox.
- **Integració amb altres funcionalitats.** Podem combinar la validació amb altres comportaments dinàmics: enviar dades amb AJAX, mostrar resums, activar botons només quan tot és correcte, etc.

Limitacions:

- **Més codi, més complexitat.** Encara que és molt més flexible, cal escriure i mantenir el codi manualment. Això pot suposar més feina i més probabilitat d'introduir errors lògics si no es fa amb cura.
- **Sense suport automàtic del navegador.** Si es desactiva JavaScript en el navegador, la validació personalitzada deixa de funcionar. Per això sempre és recomanable **complementar-la amb HTML5** o fer una validació addicional al servidor.
- **Cal més coneixements previs.** Cal entendre conceptes com funcions, condicions, valors buits i propietats dels elements del DOM abans de poder implementar validacions fiables.

2.2.3 Expressions regulars per validar cadenes

Com hem vist anteriorment, dins de la propietat `pattern` dels inputs HTML, hem escrit unes seqüències especials de caràcters, com per exemple:

```
1 pattern="^[A-Za-zÀ-ÿ-\s'\"-]{3,}$"
```

Aquestes seqüències s'anomenen **expressions regulars** (*regular expressions* o **regex**) i serveixen per **definir patrons de text** que permeten comprovar si una cadena (com un nom, correu o contrasenya) compleix un determinat format.

Les **expressions regulars** són molt útils tant a **HTML5** (amb l'atribut `pattern`) com a **JavaScript**, on podem utilitzar-les amb el constructor `RegExp` o la notació `/.../`.

Exemples d'expressions regulars habituals

Una expressió regular és un **patró que descriu una o més cadenes possibles de text**. El navegador o el codi JavaScript compara el text introduït amb aquest patró i decideix si és **vàlid** o **invàlid** (vegeu la taula 2.2).

TAULA 2.2. Expressions regulars habituals

Patró	Què valida
<code>^[A-Za-z]+\$</code>	Només lletres (sense espais ni números)
<code>^[0-9]{4}\$</code>	Exactament 4 dígits (p. ex. un PIN o any)
<code>^[A-Za-zÀ-ÿ-\s'\"-]{3,}\$</code>	Noms amb lletres, espais o guionets (mínim 3 caràcters)
<code>^[\\w\\.-]+@[\\w\\.-]+\\. [A-Za-z]{2,}\$</code>	Format bàsic d'un correu electrònic

Estructura bàsica d'una expressió regular

Una **expressió regular** és una seqüència de caràcters especials que defineixen un **patró de coincidència**. Aquest patró s'utilitza per comprovar si un text compleix una determinada forma o estructura.

Per exemple, podem validar si un camp només conté números, si comença per una lletra majúscula o si un correu té el format correcte.

Les expressions regulars es llegeixen d'**esquerra a dreta** i es componen de **caràcters literals** (que han d'aparèixer tal com són) i **metacaràcters** (que tenen un significat especial dins del patró).

Per això, en una expressió com aquesta:

```
1 ^[A-Za-zÀ-ÿ\s]{3,}$
```

Cada símbol té un **propòsit concret**:

- `^` indica que la coincidència comença al principi del text.
- `[A-Za-zÀ-ÿ\s]` defineix els caràcters permesos (en aquest cas, lletres i espais).
- `{3,}` indica que el patró s'ha de repetir com a mínim tres vegades.
- `$` marca el final de la cadena.

Així, el navegador o el codi JavaScript només acceptarà el text si **tota la cadena** compleix aquest format de principi a fi.

També és important saber que les expressions regulars són **sensibles a majúscules i minúscules** per defecte.

Si volem fer que no ho siguin, podem afegir el **modificador i** (d'*insensitive*). Per exemple:

```
1 const patro = /^[a-z]+$ /i;
```

Aquesta expressió permet tant “hola” com “HOLA”, ja que ignora les majúscules.

Finalment, convé recordar que, tot i la seva aparença complexa, les expressions regulars segueixen una **lògica molt precisa i coherent**. Amb una mica de pràctica, és possible llegir-les com si fossin frases curtes que descriuen patrons de text (“ha de començar per una lletra”, “ha de tenir 5 dígits”, “ha d'acabar amb .com”, etc.).

Això les converteix en una eina potentíssima per validar dades dins de formularis, cercar i reemplaçar text, o analitzar grans quantitats d'informació.

A la taula 2.3, podeu veure els **elements habituals de les expressions regulars**.

TAULA 2.3. Empaquetadors

Element	Significat	Exemple
<code>^</code>	Inici de la cadena	<code>^A</code> → comença per A
<code>\$</code>	Final de la cadena	<code>Z\$</code> → acaba en Z
<code>[]</code>	Conjunt permès de caràcters	<code>[A-Z]</code> → qualsevol lletra majúscula
<code>{n,m}</code>	Repeticions mínim/màxim	<code>[0-9]2,4</code> → entre 2 i 4 dígits
.		

TAULA 2.3 (continuació)

Element	Significat	Exemple
+	Una o més vegades	[a-z]+ → una o més lletres minúscules
?	Opcional (0 o 1 vegada)	colou?r → "color" o "colour"
\d	Qualsevol dígit (0–9)	\d{3} → tres díigits
\s	Espai en blanc	[A-Za-z\s]+ → lletres i espais
.	Qualsevol caràcter	a.b → "acb", "a_b", "a1b"...

Exemples en JavaScript

Anteriorment, hem vist exemples d'expressions regulars dins de l'atribut `pattern` de HTML5, però també podem fer servir expressions regulars directament al nostre codi JavaScript:

```

1  <!DOCTYPE html>
2  <html lang="ca">
3  <head>
4    <meta charset="UTF-8" />
5    <title>Validació amb expressions regulars (JS)</title>
6    <style>
7      form { border: 1px solid #ccc; padding: 15px; width: 320px; }
8      label { display: block; margin-top: 10px; }
9      #missatge { margin-top: 10px; font-weight: bold; }
10   </style>
11 </head>
12 <body>
13
14   <h3>Validació de correu amb RegExp en JavaScript</h3>
15
16   <form id="formCorreu" onsubmit="validarCorreu(); return false;">
17     <label for="email">Correu electrònic:</label>
18     <input type="text" id="email" placeholder="exemple@correu.com">
19
20     <button type="submit" style="margin-top:10px;">Validar</button>
21     <p id="missatge"></p>
22   </form>
23
24   <script>
25     function validarCorreu() {
26       const correu = document.getElementById("email").value.trim();
27       // Patró bàsic de correu: text@text.domini
28       const patroCorreu = /^[w\.-]+@[w\.-]+\.[A-Za-z]{2,}$/;
29
30       const missatge = document.getElementById("missatge");
31
32       if (!patroCorreu.test(correu)) {
33         missatge.textContent = "Introdueix un correu electrònic vàlid.";
34         missatge.style.color = "red";
35         return;
36       }
37
38       missatge.textContent = "Correu vàlid!";
39       missatge.style.color = "green";
40     }
41   </script>
42
43 </body>
44 </html>

```

El mètode `.test(correu)` retorna `true` si la cadena compleix el patró i `false` si no.

